

Start coding or generate with AI.

Bài giảng về Vibe Coding tạo code để phân tích transistor trường (FET) trong chương 8 cuốn sách trên

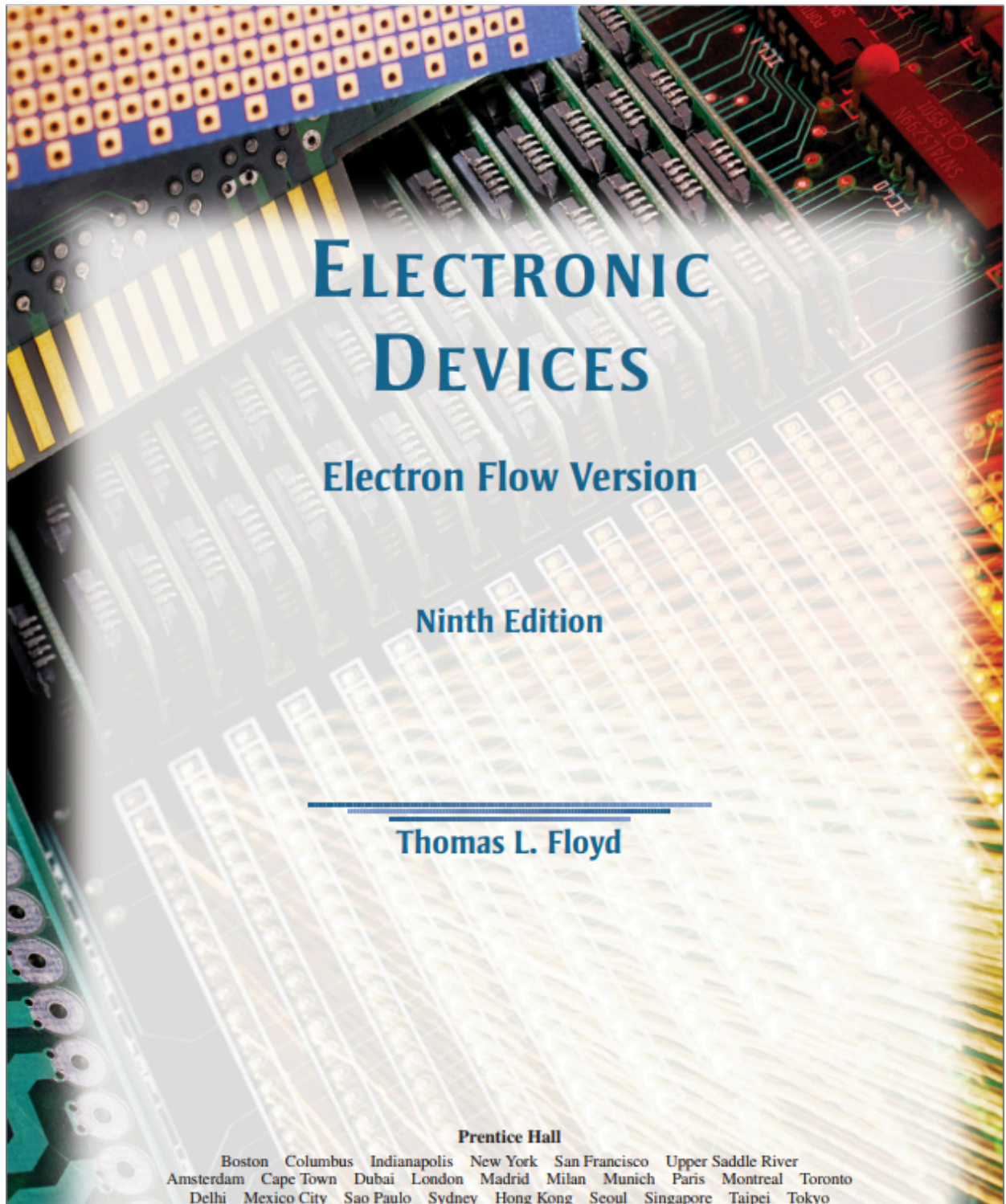
Sưu tầm và Biên soạn bổ sung phần Vibe Coding

With the support of Gemini and A/ Prof. Huy Trinh Le, UIT, 2026

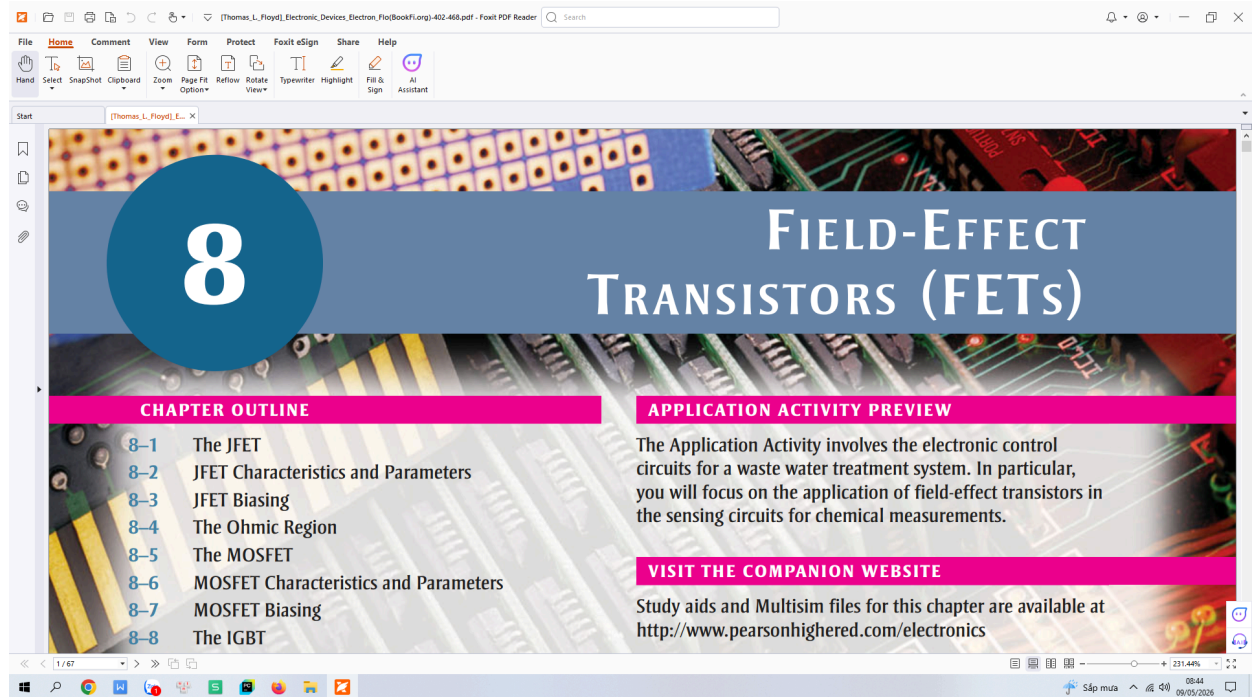
Prof Happy AI

Date: May 9, 2026

✓ Book: Electronic Devices, Electron Flow Version, 2012



Chapter 8:



Các chương trình (như Python, Matlab hoặc các công cụ mô phỏng mạch) tập trung vào các dạng sau:

1. Tính toán các thông số đặc trưng của JFET và D-MOSFET Các bài tập này yêu cầu xác định dòng điện hoặc điện áp dựa trên phương trình đặc tuyến truyền dẫn (Square-law): Tính dòng điện máng (I_D): Sử dụng công thức $I_D = I_{DSS}(1 - \frac{V_{GS}}{V_{GS(off)}})^2$. Xác định điện áp ngắt ($V_{GS(off)}$) hoặc dòng điện bão hòa (I_{DSS}): Khi biết các giá trị thực nghiệm khác. Tính độ dẫn (g_m): Tính g_m tại một điểm làm việc cụ thể bằng công thức $g_m = g_{m0}(1 - \frac{V_{GS}}{V_{GS(off)}})$.
2. Phân tích điểm làm việc tĩnh (Q-point) Bạn có thể lập trình để giải các mạch định thiên (bias) khác nhau: Tự định thiên (Self-Bias): Giải phương trình bậc hai kết hợp giữa đường thẳng định thiên $V_{GS} = -I_D R_S$ và phương trình đặc tuyến của JFET. Định thiên bằng cầu phân áp (Voltage-Divider Bias): Tính toán V_G , sau đó xác định I_D và V_{GS} để tìm điểm Q. Định thiên bằng nguồn dòng (Current-Source Bias): Xác định các giá trị điện trở R_E và R_D để đạt được dòng điện mong muốn.
3. Bài tập về E-MOSFET Xác định dòng điện cho E-MOSFET: Sử dụng công thức $I_D = K(V_{GS} - V_{GS(th)})^2$. Tính hằng số K: Dựa trên các thông số từ bảng dữ liệu (datasheet) như $I_{D(on)}$ tại một giá trị V_{GS} nhất định.
4. Phân tích vùng Ohm (Ohmic Region) Tính điện trở máng-nguồn (r_{ds}): Sử dụng JFET như một điện trở thay đổi được bằng điện áp, tính toán giá trị điện trở dựa trên V_{GS} .
5. Xử lý lỗi và mô phỏng (Troubleshooting) Lập trình mô phỏng các lỗi mạch: Ví dụ như tính toán sự thay đổi của điện áp máng (V_D) khi một điện trở bị

hở mạch hoặc ngắn mạch. Ví dụ cụ thể từ tài liệu: Ví dụ 8-4: Xác định độ hỗ dẫn g_m cho JFET 2N5457 tại $V_{GS} = -4V$.

Ví dụ 8-10: Xác định điểm Q cho mạch JFET tự định thiên. Ví dụ 8-20: Xác định điện áp máng-nguồn (V_{DS}) trong mạch D-MOSFET định thiên bằng không (zero-biased).

Các công thức toán học này rất phù hợp để chuyển đổi thành mã nguồn nhằm tự động hóa việc tính toán và thiết kế mạch.

Để lập trình giải các bài tập về transistor trường (FET) trong tài liệu này, chúng ta sử dụng các mảng kiến thức Toán học từ cơ bản đến ứng dụng như sau:

1. Đại số và Giải phương trình: Đây là phần quan trọng nhất để tìm điểm làm việc (Q-point) của JFET và MOSFET: Phương trình bậc hai (Quadratic Equations): Công thức Shockley $I_D = I_{DSS} \left(1 - \frac{V_{GS}}{V_{GS(off)}}\right)^2$ thực chất là một phương trình bậc hai theo biến V_{GS} hoặc I_D . Khi giải mạch tự định thiên (Self-bias), bạn phải kết hợp phương trình này với phương trình đường thẳng định thiên ($V_{GS} = -I_D R_S$) để tìm nghiệm giao điểm. Hệ phương trình tuyến tính: Sử dụng khi phân tích các mạch phức tạp hơn có nhiều vòng lặp (mesh) hoặc nút (node) dựa trên định luật Kirchhoff.
2. Hàm số và Đồ thị (Analytical Geometry): Hàm phi tuyến: Đặc tuyến truyền dẫn của FET không phải là đường thẳng mà là một phần của đường Parabol. Việc lập trình cần hiểu cách biểu diễn hàm số này trên hệ trục tọa độ $I_D - V_{GS}$. Độ dốc của hàm số (Slope): Độ hỗ dẫn (g_m) chính là độ dốc của đường đặc tuyến truyền dẫn tại một điểm cụ thể. Điểm giao cắt (Intersections): Giải bài toán bằng lập trình chính là tìm giao điểm giữa đường cong phi tuyến (đặc tuyến FET) và đường thẳng (đường định thiên - bias line).
3. Giải tích (Calculus) Đạo hàm (Derivatives): Độ hỗ dẫn g_m được định nghĩa về mặt toán học là đạo hàm của dòng điện máng theo điện áp cổng-nguồn: $g_m = \frac{dI_D}{dV_{GS}}$. Từ công thức Shockley, khi lấy đạo hàm, ta có: $g_m = g_{m0} \left(1 - \frac{V_{GS}}{V_{GS(off)}}\right)$, trong đó $g_{m0} = \frac{-2I_{DSS}}{V_{GS(off)}}$. Việc hiểu đạo hàm giúp bạn lập trình tính toán sự thay đổi nhỏ của tín hiệu (phân tích xoay chiều AC).
4. Toán rời rạc và Giải thuật số (Numerical Methods): Nếu bạn không giải phương trình bằng công thức trực tiếp (đại số), bạn có thể lập trình sử dụng các phương pháp số: Phương pháp Newton-Raphson: Để tìm nghiệm của các phương trình phi tuyến phức tạp trong mô phỏng mạch. Phép xấp xỉ (Approximation): Tính toán các giá trị trong vùng Ohmic khi FET hoạt động như một điện trở biến thiên

theo điện áp. 5. Thống kê và Xử lý sai số Tính toán cận trên/cận dưới (Min/Max Analysis): Trong tài liệu thường cho các giá trị I_{DSS} và $V_{GS(off)}$ dưới dạng một khoảng (ví dụ: từ -2V đến -8V). Lập trình cần sử dụng các phép toán thống kê hoặc xét trường hợp biên để xác định dải hoạt động của mạch. Tóm tắt ứng dụng lập trình: Nếu bạn dùng Python, các thư viện như NumPy sẽ hỗ trợ giải hệ phương trình, còn Matplotlib giúp bạn vẽ đồ thị đường định thiên cắt đường đặc tuyến để trực quan hóa kết quả bài tập.

▼ Chương trình mẫu 1:

Chương trình Tính toán các thông số đặc trưng của JFET và D-MOSFET Các bài tập này yêu cầu xác định dòng điện hoặc điện áp dựa trên phương trình đặc tuyến truyền dẫn (Square-law): Tính dòng điện máng (I_D): Sử dụng công thức $I_D = I_{DSS}(1 - V_{GS}/V_{GS(off)})^2$. Xác định điện áp ngắt ($V_{GS(off)}$) hoặc dòng điện bão hòa (I_{DSS}): Khi biết các giá trị thực nghiệm khác. Tính độ dẫn (gm): Tính gm tại một điểm làm việc cụ thể bằng công thức $g_m = g_{m0}(1 - V_{GS}/V_{GS(off)})$.

```
import math

def calculate_id(idss, vgs_off, vgs):
    """
    Tính dòng điện máng ID dựa trên công thức Square-law.
    ID = IDSS * (1 - VGS / VGS_off)^2
    """
    if vgs / vgs_off > 1 or (vgs < 0 and vgs_off > 0) or (vgs > 0 and vgs_off < 0):
        # Nếu VGS nằm ngoài vùng hoạt động (vượt quá điểm ngắt)
        return 0.0
    id_current = idss * (1 - vgs / vgs_off)**2
    return id_current

def calculate_gm(idss, vgs_off, vgs):
    """
    Tính độ dẫn gm tại một điểm VGS cụ thể.
    gm0 = 2 * IDSS / |VGS_off|
    gm = gm0 * (1 - VGS / VGS_off)
    """
    gm0 = (2 * idss) / abs(vgs_off)
    gm = gm0 * (1 - vgs / vgs_off)
    return gm

def calculate_vgs_from_id(idss, vgs_off, id_target):
    """
    Xác định VGS khi biết ID (Giải phương trình ngược).
    VGS = VGS_off * (1 - sqrt(ID / IDSS))
    """
    vgs = vgs_off * (1 - math.sqrt(id_target / idss))
    return vgs
```

```
# --- VÍ DỤ ÁP DỤNG ---

# Giả sử thông số từ datasheet của một JFET:
IDSS_val = 10e-3    # 10 mA
VGS_off_val = -4.0  # -4 V
VGS_work = -2.0     # Điểm làm việc hiện tại là -2V

print("--- KẾT QUẢ TÍNH TOÁN FET ---")

# 1. Tính ID
id_res = calculate_id(IDSS_val, VGS_off_val, VGS_work)
print(f"1. Với VGS = {VGS_work}V, dòng điện máng ID = {id_res*1000:.2f} mA")

# 2. Tính gm
gm_res = calculate_gm(IDSS_val, VGS_off_val, VGS_work)
print(f"2. Độ hỗ dẫn gm tại điểm đó = {gm_res*1000:.2f} mS (mili-Siemens)")

# 3. Tính ngược VGS khi biết ID mong muốn
target_id = 5e-3 # Giả sử muốn tìm VGS để có ID = 5mA
vgs_needed = calculate_vgs_from_id(IDSS_val, VGS_off_val, target_id)
print(f"3. Để có ID = {target_id*1000:.1f}mA, cần đặt VGS xấp xỉ = {vgs_needed}V")

--- KẾT QUẢ TÍNH TOÁN FET ---
1. Với VGS = -2.0V, dòng điện máng ID = 2.50 mA
2. Độ hỗ dẫn gm tại điểm đó = 2.50 mS (mili-Siemens)
3. Để có ID = 5.0mA, cần đặt VGS xấp xỉ = -1.17 V
```

Giải thích các thành phần trong Code:

Hàm `calculate_id`: Áp dụng đúng công thức $I_D = I_{DSS} \left(1 - \frac{V_{GS}}{V_{GS(off)}}\right)^2$.

Tôi có thêm điều kiện kiểm tra nếu V_{GS} vượt quá điện áp ngắt thì dòng điện sẽ bằng 0.

Hàm `calculate_gm`: Đầu tiên tính độ hỗ dẫn cực đại g_{m0} (thường xảy ra khi $V_{GS} = 0$), sau đó tính g_m tại điểm làm việc mong muốn. Đây là kiến thức giải tích về đạo hàm của đường cong đặc tuyến.

Hàm `calculate_vgs_from_id`: Đây là kỹ thuật biến đổi đại số để tìm giá trị thực nghiệm. Nó giúp bạn giải các bài tập yêu cầu tìm mức điện áp cổng cần thiết để đạt được một dòng điện mục tiêu.

Lưu ý về đơn vị: Trong lập trình, tốt nhất bạn nên quy đổi tất cả về đơn vị chuẩn (Ampe, Volt, Siemens). Khi in kết quả ra màn hình, ta nhân với 1000 để hiển thị đơn vị mA hoặc mS cho dễ đọc giống như trong các giáo trình điện tử.

✓ Biểu diễn trực quan:

Việc sử dụng matplotlib để biểu diễn trực quan đặc tuyến truyền dẫn (Transfer Characteristic Curve) là cách tốt nhất để hiểu về điểm làm việc của JFET. Đồ thị này sẽ giúp bạn thấy rõ mối quan hệ phi tuyến giữa V_{GS} và I_D . Dưới đây là mã Python bổ sung thư viện numpy và matplotlib để vẽ đồ thị dựa trên các hàm bạn đã viết:

```

import math
import numpy as np
import matplotlib.pyplot as plt

# --- Giữ nguyên các hàm tính toán của bạn ---
def calculate_id(idss, vgs_off, vgs):
    if (vgs_off < 0 and vgs < vgs_off) or (vgs_off > 0 and vgs > vgs_off):
        return 0.0
    return idss * (1 - vgs / vgs_off)**2

# --- Cấu hình thông số ---
IDSS_val = 10e-3    # 10 mA
VGS_off_val = -4.0  # -4 V
VGS_work = -2.0     # Điểm làm việc (Q-point)

# 1. Tạo dữ liệu cho đường đặc tuyến
# Tạo mảng VGS từ VGS_off đến 0 (cho JFET kênh N)
vgs_array = np.linspace(VGS_off_val, 0, 100)
id_array = np.array([calculate_id(IDSS_val, VGS_off_val, v) for v in vgs_array])

# 2. Tính toán điểm làm việc hiện tại (Q-point)
id_work = calculate_id(IDSS_val, VGS_off_val, VGS_work)

# 3. Vẽ đồ thị
plt.figure(figsize=(10, 6))
plt.plot(vgs_array, id_array * 1000, label='Đặc tuyến truyền dẫn (Shockley)')

# Đánh dấu điểm Q-point
plt.plot(VGS_work, id_work * 1000, 'ro', label=f'Q-point (VGS={VGS_work}V, I')

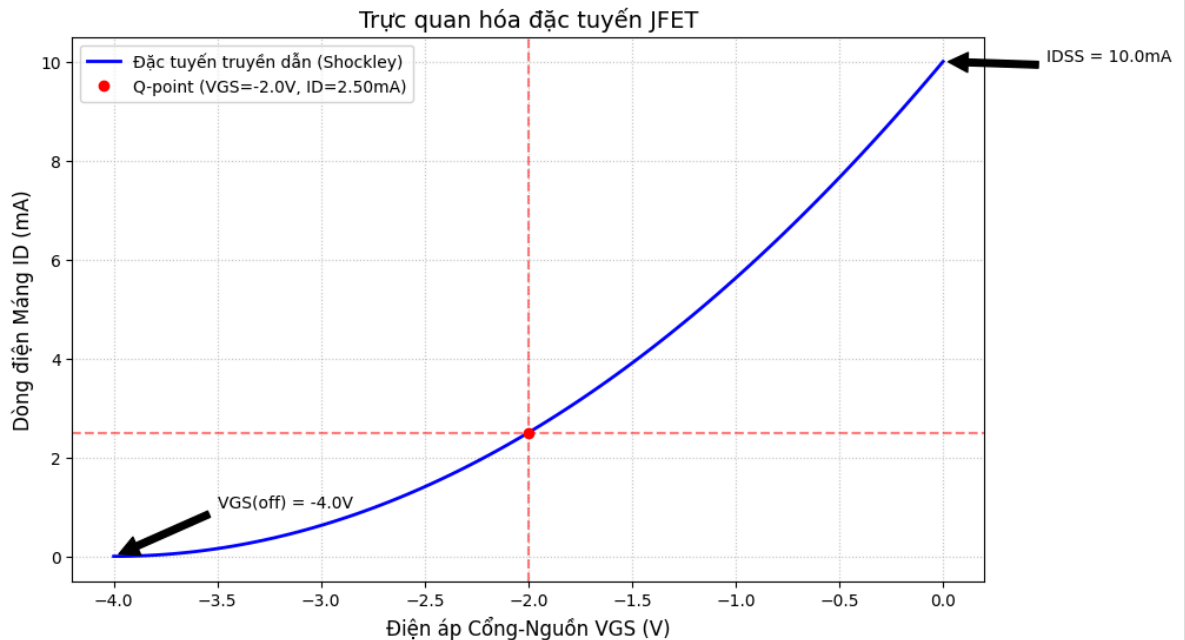
# Vẽ các đường giống cho điểm Q
plt.axvline(x=VGS_work, color='red', linestyle='--', alpha=0.5)
plt.axhline(y=id_work * 1000, color='red', linestyle='--', alpha=0.5)

# Cấu hình tiêu đề và trục
plt.title('Trực quan hóa đặc tuyến JFET', fontsize=14)
plt.xlabel('Điện áp Cổng-Nguồn VGS (V)', fontsize=12)
plt.ylabel('Dòng điện Máng ID (mA)', fontsize=12)
plt.grid(True, linestyle=':', alpha=0.7)
plt.legend()

# Thêm ghi chú thông số IDSS và VGS(off)
plt.annotate(f'IDSS = {IDSS_val*1000}mA', xy=(0, IDSS_val*1000), xytext=(0.5,
    arrowprops=dict(facecolor='black', shrink=0.05))
plt.annotate(f'VGS(off) = {VGS_off_val}V', xy=(VGS_off_val, 0), xytext=(VGS_
    arrowprops=dict(facecolor='black', shrink=0.05))

plt.show()

```



▼ Chương trình mẫu 2:

Chương trình Phân tích điểm làm việc tĩnh (Q-point) Bạn có thể lập trình để giải các mạch định thiên (bias) khác nhau:

- Tự định thiên (Self-Bias): Giải phương trình bậc hai kết hợp giữa đường thẳng định thiên $V_{GS} = -I_D R_S$ và phương trình đặc tuyến của JFET.
- Định thiên bằng cầu phân áp (Voltage-Divider Bias): Tính toán V_G , sau đó xác định I_D và V_{GS} để tìm điểm Q.
- Định thiên bằng nguồn dòng (Current-Source Bias): Xác định các giá trị điện trở R_E và R_D để đạt được dòng điện mong muốn.

Để lập trình giải điểm Q-point cho các mạch định thiên JFET, chúng ta cần kết hợp giữa các định luật Kirchhoff (mô tả mạch điện) và phương trình Shockley (mô tả linh kiện).

Dưới đây là đoạn mã Python giải quyết 3 cấu hình mạch phổ biến nhất:

```
import math
```

```

def solve_self_bias(idss, vgs_off, rs):
    """
    Giải mạch Tự định thiên (Self-Bias) cho JFET/D-MOSFET.
    Phương trình:  $(ID_{SS} * R_S / V_{GS\_off}^2) * V_{GS}^2 + (1 - 2 * ID_{SS} * R_S / V_{GS\_off}) * V_{GS} +$ 
    """
    # Hệ số phương trình bậc 2:  $a * V_{GS}^2 + b * V_{GS} + c = 0$ 
    a = (idss * rs) / (vgs_off**2)
    b = 1 - (2 * idss * rs / vgs_off)
    c = idss * rs

    delta = b**2 - 4*a*c
    if delta < 0:
        return None

    # Tính 2 nghiệm của VGS
    vgs1 = (-b + math.sqrt(delta)) / (2 * a)
    vgs2 = (-b - math.sqrt(delta)) / (2 * a)

    # Nghiệm hợp lý phải nằm trong khoảng  $[V_{GS\_off}, 0]$  (đối với kênh N)
    # hoặc  $[0, V_{GS\_off}]$  (đối với kênh P)
    vgs_min, vgs_max = sorted([vgs_off, 0])
    vgs_q = vgs1 if vgs_min <= vgs1 <= vgs_max else vgs2

    id_q = -vgs_q / rs
    return id_q, vgs_q

def solve_voltage_divider(idss, vgs_off, vdd, r1, r2, rs):
    """
    Giải mạch Cầu phân áp (Voltage-Divider Bias).
    Phương trình:  $(K * R_S^2) ID^2 - (2 * K * R_S * T + 1) ID + K * T^2 = 0$  với  $T = V_G - V_{GS\_off}$ 
    """
    vg = vdd * (r2 / (r1 + r2))
    k = idss / (vgs_off**2)
    t = vg - vgs_off

    a = k * (rs**2)
    b = -(2 * k * rs * t + 1)
    c = k * (t**2)

    delta = b**2 - 4*a*c
    if delta < 0: return None

    id1 = (-b + math.sqrt(delta)) / (2 * a)
    id2 = (-b - math.sqrt(delta)) / (2 * a)

    # Kiểm tra nghiệm ID nào cho  $V_{GS} > V_{GS\_off}$ 
    vgs1 = vg - id1 * rs
    id_q = id1 if vgs1 > vgs_off else id2
    vgs_q = vg - id_q * rs

    return id_q, vgs_q

def design_current_source_bias(target_id, v_ee, vbe=0.7):
    """Thiết kế RE cho mạch nguồn dòng"""
    re = (abs(v_ee) - vbe) / target_id

```

```

return re

# --- KIỂM TRA VỚI DỮ LIỆU MẪU ---
print("--- KẾT QUẢ PHÂN TÍCH Q-POINT ---")

# Thông số JFET (Ví dụ: 2N5457)
IDSS = 12e-3 # 12 mA
VGS_OFF = -5.0 # -5 V

# 1. Test Self-Bias với RS = 500 Ohm
res_s = solve_self_bias(IDSS, VGS_OFF, 500)
if res_s:
    print(f"[Self-Bias] IDq = {res_s[0]*1000:.2f} mA, VGSq = {res_s[1]:.2f}")

# 2. Test Voltage-Divider (VDD=12V, R1=6.8M, R2=1M, RS=1k)
res_v = solve_voltage_divider(IDSS, VGS_OFF, 12, 6.8e6, 1e6, 1000)
if res_v:
    print(f"[Voltage-Divider] IDq = {res_v[0]*1000:.2f} mA, VGSq = {res_v[1]:.2f}")

# 3. Test Current-Source Bias
re_needed = design_current_source_bias(2e-3, 5.0)
print(f"[Current-Source] Để ID = 2mA, chọn RE = {re_needed:.0f} Ohm")

--- KẾT QUẢ PHÂN TÍCH Q-POINT ---
[Self-Bias] IDq = 4.13 mA, VGSq = -2.07 V
[Voltage-Divider] IDq = 3.75 mA, VGSq = -2.21 V
[Current-Source] Để ID = 2mA, chọn RE = 2150 Ohm

```

Giải thích code: Đoạn mã Python trên được thiết kế để mô phỏng lại cách một kỹ sư điện tử phân tích mạch bằng toán học. Thay vì vẽ đồ thị bằng tay, chúng ta sử dụng các thuật toán giải phương trình để tìm Điểm làm việc tĩnh (Q-point) – tức là giá trị dòng điện I_D và điện áp V_{GS} khi mạch chưa có tín hiệu xoay chiều. Dưới đây là giải thích chi tiết từng phần:

1. Hàm `solve_self_bias` (Mạch tự định thiên) Đây là mạch dùng một điện trở R_S nối từ cực Nguồn (Source) xuống đất để tạo ra điện áp âm cho cực Cổng (Gate). Logic Toán học: Chúng ta phải giải hệ phương trình gồm: Phương trình linh kiện (Shockley): $I_D = I_{DSS} \left(1 - \frac{V_{GS}}{V_{GS(off)}}\right)^2$ Phương trình mạch: $V_{GS} = -I_D R_S$ Cách code xử lý: Thay I_D từ phương trình (2) vào (1), ta được một phương trình bậc hai chỉ còn biến V_{GS} . Hệ số a, b, c : Được rút gọn từ việc khai triển đa thức. Chọn nghiệm: Phương trình bậc hai luôn cho 2 nghiệm. Tuy nhiên, JFET kênh N chỉ hoạt động khi V_{GS} nằm trong khoảng từ $V_{GS(off)}$ đến $0V$. Code sử dụng hàm `sorted()` và so sánh điều kiện để loại bỏ nghiệm "ảo" không có thực trong vật lý.
2. Hàm `solve_voltage_divider` (Mạch cầu phân áp) Đây là mạch phổ biến nhất vì nó giúp điểm Q ổn định hơn trước sự thay đổi của linh kiện. Bước 1 - Tính V_G : Sử dụng công thức chia áp $V_G = V_{DD} \cdot \frac{R_2}{R_1 + R_2}$. Đây là điện áp cố định tại cực

Cổng.Bước 2 - Thiết lập phương trình: $V_{GS} = V_G - I_D R_S$. Cách code xử lý: Tương tự như mạch tự định thiên, code biến đổi đại số để đưa về dạng $a \cdot I_D^2 + b \cdot I_D + c = 0$. Ở đây code giải trực tiếp để tìm dòng điện I_D trước, sau đó mới suy ra V_{GS} . Biễn t: Tôi đặt $t = v_g - v_{gs_off}$ để làm gọn công thức khi tính các hệ số a, b, c .

3. Hàm `design_current_source_bias` (Thiết kế nguồn dòng) Phần này không giải phương trình bậc hai mà giải bài toán thiết kế. Nguyên lý: Mạch này dùng một transistor BJT ở phía dưới để "ép" dòng điện qua JFET phải bằng một giá trị cố định, bất kể đặc tính của JFET đó như thế nào. Tính toán: Dòng điện I_D xấp xỉ bằng dòng điện cực phát I_E của BJT. Theo định luật Ohm: $R_E = \frac{V_{EE} - V_{BE}}{I_D}$. Code chỉ đơn giản thực hiện phép chia này để báo cho bạn biết cần chọn điện trở R_E bao nhiêu Ohm.
4. Các điểm kỹ thuật cần lưu ý trong Code: Đơn vị: Code sử dụng đơn vị chuẩn (Ampe, Volt, Ohm). Do đó, khi nhập $12mA$, bạn phải viết là $12e-3$. Kiểm tra Delta (Δ): Trước khi lấy căn bậc hai (`math.sqrt`), code luôn kiểm tra `if delta < 0`. Điều này ngăn chương trình bị lỗi nếu bạn nhập thông số mạch không thực tế (mạch không thể hoạt động). Tính linh hoạt: Code có thể giải cho cả JFET và MOSFET loại hiếm (D-MOSFET) vì chúng dùng chung phương trình Square-law. Việc sử dụng code này giúp bạn kiểm tra nhanh kết quả của các bài tập trong chương 8 mà không cần phải thực hiện các phép tính căn thức phức tạp bằng tay.

✓ Biểu diễn trực quan:

Để trực quan hóa các phương pháp định thiên (Bias) khác nhau, chúng ta sẽ sử dụng `matplotlib` để vẽ đồng thời đường đặc tuyến truyền dẫn của JFET và các đường định thiên (bias lines) tương ứng. Điểm giao nhau giữa chúng chính là điểm Q (Q-point).

Dưới đây là mã nguồn mở rộng để vẽ đồ thị cho cả hai trường hợp Self-Bias và Voltage-Divider Bias:

```
import math
import numpy as np
import matplotlib.pyplot as plt

def solve_self_bias(idss, vgs_off, rs):
    a = (idss * rs) / (vgs_off**2)
    b = 1 - (2 * idss * rs / vgs_off)
    c = idss * rs
    delta = b**2 - 4*a*c
    if delta < 0: return None
    vgs1 = (-b + math.sqrt(delta)) / (2 * a)
    vgs2 = (-b - math.sqrt(delta)) / (2 * a)
    vgs_min, vgs_max = sorted([vgs_off, 0])
```

```

vgs_q = vgs1 if vgs_min <= vgs1 <= vgs_max else vgs2
id_q = -vgs_q / rs
return id_q, vgs_q

def solve_voltage_divider(idss, vgs_off, vdd, r1, r2, rs):
    vg = vdd * (r2 / (r1 + r2))
    k = idss / (vgs_off**2)
    t = vg - vgs_off
    a = k * (rs**2)
    b = -(2 * k * rs * t + 1)
    c = k * (t**2)
    delta = b**2 - 4*a*c
    if delta < 0: return None
    id1 = (-b + math.sqrt(delta)) / (2 * a)
    id2 = (-b - math.sqrt(delta)) / (2 * a)
    vgs1 = vg - id1 * rs
    id_q = id1 if vgs1 > vgs_off else id2
    vgs_q = vg - id_q * rs
    return id_q, vgs_q, vg

# --- Cấu hình thông số ---
IDSS = 12e-3
VGS_OFF = -5.0

# Tạo dữ liệu đường đặc tuyến Shockley
vgs_curve = np.linspace(VGS_OFF, 0, 100)
id_curve = IDSS * (1 - vgs_curve / VGS_OFF)**2

# 1. Tính toán cho Self-Bias (RS = 500)
rs_s = 500
id_qs, vgs_qs = solve_self_bias(IDSS, VGS_OFF, rs_s)

# 2. Tính toán cho Voltage-Divider (VDD=12, R1=6.8M, R2=1M, RS=1k)
vdd, r1, r2, rs_v = 12, 6.8e6, 1e6, 1000
id_qv, vgs_qv, vg_v = solve_voltage_divider(IDSS, VGS_OFF, vdd, r1, r2, rs_v)

# --- VẼ ĐỒ THỊ ---
plt.figure(figsize=(12, 7))

# Vẽ đặc tuyến truyền dẫn chung
plt.plot(vgs_curve, id_curve * 1000, 'k-', label='JFET Characteristic', line

# --- TRƯỜNG HỢP 1: SELF-BIAS ---
# Đường định thiên: ID = -VGS / RS => qua (0,0) và (VGS_off, -VGS_off/RS)
vgs_line_s = np.array([VGS_OFF, 0])
id_line_s = -vgs_line_s / rs_s
plt.plot(vgs_line_s, id_line_s * 1000, 'r--', label=f'Self-Bias Line (RS={rs_s}')
plt.plot(vgs_qs, id_qs * 1000, 'ro')
plt.annotate(f'Qs({vgs_qs:.2f}V, {id_qs*1000:.2f}mA)', (vgs_qs, id_qs*1000),

# --- TRƯỜNG HỢP 2: VOLTAGE-DIVIDER ---
# Đường định thiên: ID = (VG - VGS) / RS => qua (VG, 0) và (VGS_off, (VG-VGS)/RS)
vgs_line_v = np.array([VGS_OFF, vg_v])
id_line_v = (vg_v - vgs_line_v) / rs_v
plt.plot(vgs_line_v, id_line_v * 1000, 'g--', label=f'Voltage-Divider Line (
```

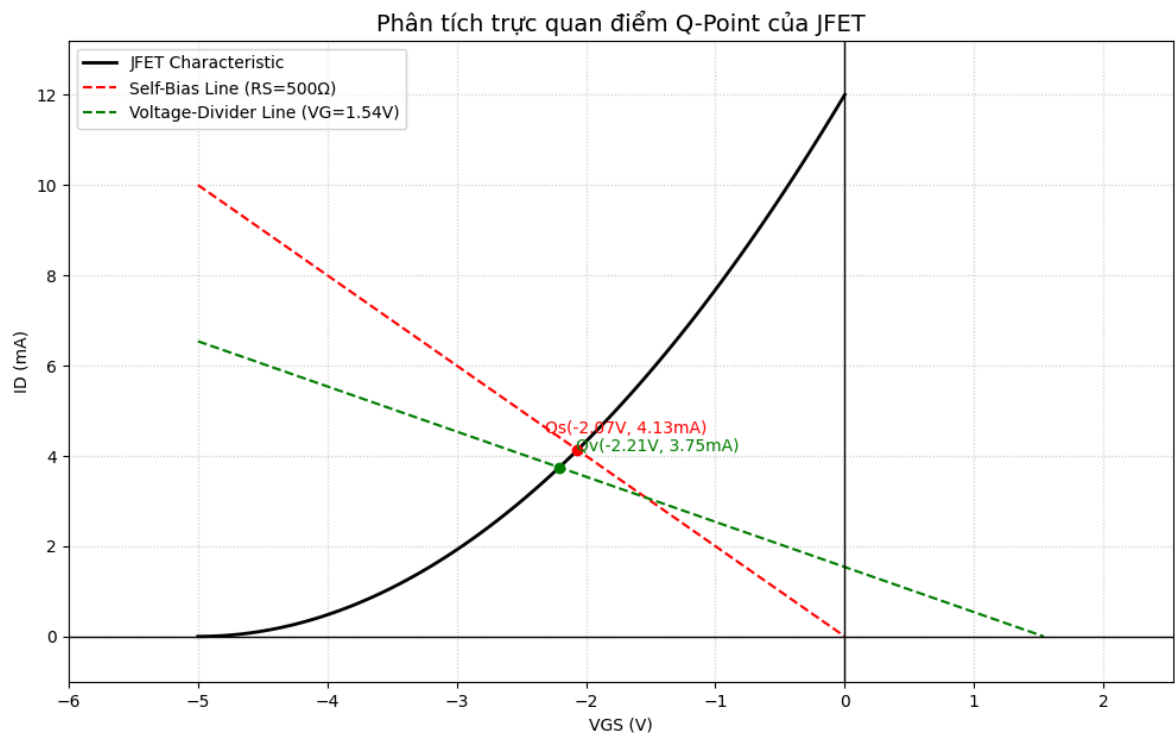
```

plt.plot(vgs_qv, id_qv * 1000, 'go')
plt.annotate(f'Qv({vgs_qv:.2f}V, {id_qv*1000:.2f}mA)', (vgs_qv, id_qv*1000),

# Tùy chỉnh đồ thị
plt.axhline(0, color='black', lw=1)
plt.axvline(0, color='black', lw=1)
plt.title('Phân tích trực quan điểm Q-Point của JFET', fontsize=14)
plt.xlabel('VGS (V)')
plt.ylabel('ID (mA)')
plt.legend()
plt.grid(True, linestyle=':', alpha=0.6)
plt.ylim(-1, (IDSS*1.1)*1000)
plt.xlim(VGS_OFF - 1, vgs_v + 1)

plt.show()

```



✓ Chương trình mẫu 3:

Bài tập về E-MOSFET Xác định dòng điện cho E-MOSFET: Sử dụng công thức $I_D = K(V_{GS} - V_{GS(th)})^2$. Tính hằng số K: Dựa trên các thông số từ bảng dữ liệu (datasheet) như $I_{D(on)}$ tại một giá trị V_{GS} nhất định.

Khác với JFET hay D-MOSFET, dòng E-MOSFET (Enhancement MOSFET) chỉ bắt đầu dẫn điện khi điện áp cổng-nguồn (V_{GS}) vượt quá một giá trị ngưỡng nhất định ($V_{GS(th)}$). Dưới đây là đoạn mã Python giúp bạn tính toán hằng số định mức K và dòng điện máng I_D cho E-MOSFET:

```
import math

def calculate_k_constant(id_on, vgs_on, vgs_th):
    """
    Tính hằng số K của E-MOSFET dựa trên thông số định mức từ Datasheet.
    Công thức: K = ID(on) / (VGS_on - VGS_th)^2
    """
    denominator = (vgs_on - vgs_th) ** 2
    if denominator == 0:
        return 0
    k = id_on / denominator
    return k

def calculate_id_emosfet(k, vgs, vgs_th):
    """
    Tính dòng điện máng ID cho E-MOSFET.
    Công thức: ID = K * (VGS - VGS_th)^2
    Lưu ý: ID = 0 nếu VGS < VGS_th
    """
    if vgs <= vgs_th:
        return 0.0
    id_current = k * (vgs - vgs_th) ** 2
    return id_current

# --- VÍ DỤ ÁP DỤNG ---
print("---- PHÂN TÍCH E-MOSFET ----")

# Thông số giả định từ Datasheet (Ví dụ loại 2N7000)
id_on_spec = 75e-3      # ID(on) = 75mA
vgs_on_spec = 4.5       # Tại VGS = 4.5V
vgs_threshold = 2.0     # VGS(th) = 2.0V

# Bước 1: Tính hằng số K
k_val = calculate_k_constant(id_on_spec, vgs_on_spec, vgs_threshold)
print(f"1. Hằng số K tính được: {k_val:.6f} A/V^2")

# Bước 2: Tính dòng ID tại một mức áp VGS cụ thể
vgs_test = 5.0 # Giả sử đặt VGS = 5V
id_res = calculate_id_emosfet(k_val, vgs_test, vgs_threshold)

print(f"2. Với VGS = {vgs_test}V, dòng điện máng ID = {id_res*1000:.2f} mA")
```

```
# Bước 3: Kiểm tra trường hợp VGS thấp hơn ngưỡng
vgs_low = 1.5
id_low = calculate_id_emosfet(k_val, vgs_low, vgs_threshold)
print(f"3. Với VGS = {vgs_low}V (thấp hơn ngưỡng {vgs_threshold}V), ID = {id
```

```
--- PHÂN TÍCH E-MOSFET ---
1. Hằng số K tính được: 0.012000 A/V^2
2. Với VGS = 5.0V, dòng điện máng ID = 108.00 mA
3. Với VGS = 1.5V (thấp hơn ngưỡng 2.0V), ID = 0.0 mA
```

Giải thích các thành phần Toán học trong Code:Hàm calculate_k_constant:Trong bảng dữ liệu của E-MOSFET, nhà sản xuất thường không cho sẵn K . Thay vào đó, họ cho một điểm dữ liệu cụ thể gọi là $I_{D(on)}$ tại một mức $V_{GS(on)}$.Code sử dụng phép biến đổi: $K = \frac{I_{D(on)}}{(V_{GS(on)} - V_{GS(th)})^2}$. Đây là bước chuẩn bị quan trọng trước khi có thể tính toán bất kỳ giá trị nào khác.Hàm calculate_id_emosfet:Đây là hàm thực hiện phương trình đặc tuyến chính của E-MOSFET.Logic điều kiện (if vgs <= vgs_th): Đây là điểm mấu chốt. E-MOSFET hoạt động ở chế độ "tăng cường", nghĩa là kênh dẫn chỉ được hình thành khi V_{GS} đủ lớn để hút các hạt mang điện tạo thành kênh. Nếu điện áp thấp hơn ngưỡng ($V_{GS(th)}$), thiết bị sẽ ở trạng thái ngắt (OFF) và dòng điện bằng 0.Ứng dụng:Bạn có thể sử dụng đoạn mã này để vẽ đường đặc tuyến truyền dẫn của MOSFET bằng cách cho V_{GS} chạy trong một vòng lặp từ 0V đến V_{DD} và lưu lại các giá trị I_D tương ứng.

✓ Biểu diễn trực quan:

Để trực quan hóa đặc tính của E-MOSFET, chúng ta sẽ sử dụng matplotlib để vẽ đường đặc tuyến truyền dẫn. Điểm khác biệt quan trọng nhất của E-MOSFET so với JFET là dòng điện bằng 0 cho đến khi V_{GS} vượt qua điện áp ngưỡng $V_{GS(th)}$.Dưới đây là mã Python để vẽ đồ thị này:

```
import math
import numpy as np
import matplotlib.pyplot as plt

def calculate_k_constant(id_on, vgs_on, vgs_th):
    denominator = (vgs_on - vgs_th) ** 2
    return id_on / denominator if denominator != 0 else 0

def calculate_id_emosfet(k, vgs, vgs_th):
    if vgs <= vgs_th:
        return 0.0
    return k * (vgs - vgs_th) ** 2

# --- Thông số giả định (2N7000) ---
id_on_spec = 75e-3
vgs_on_spec = 4.5
```

```
vgs_threshold = 2.0
vgs_test = 5.0

# Tính toán các giá trị cần thiết
k_val = calculate_k_constant(id_on_spec, vgs_on_spec, vgs_threshold)
id_test = calculate_id_emosfet(k_val, vgs_test, vgs_threshold)

# 1. Tạo dữ liệu cho đường đặc tuyến
# Vẽ từ 0V đến mức VGS cao hơn VGS(on) một chút để thấy rõ độ dốc
vgs_range = np.linspace(0, 6, 200)
id_range = np.array([calculate_id_emosfet(k_val, v, vgs_threshold) for v in

# 2. Vẽ đồ thị
plt.figure(figsize=(10, 6))
plt.plot(vgs_range, id_range * 1000, label='Đặc tuyến E-MOSFET', color='purp

# Đánh dấu vùng ngắt (Cutoff region)
plt.fill_between(vgs_range, 0, id_range * 1000, where=(vgs_range <= vgs_thre
                color='gray', alpha=0.2, label='Vùng ngắt (ID=0)')

# Đánh dấu điểm kiểm tra (Test point)
plt.plot(vgs_test, id_test * 1000, 'ro', label=f'Điểm thử nghiệm ({vgs_test}

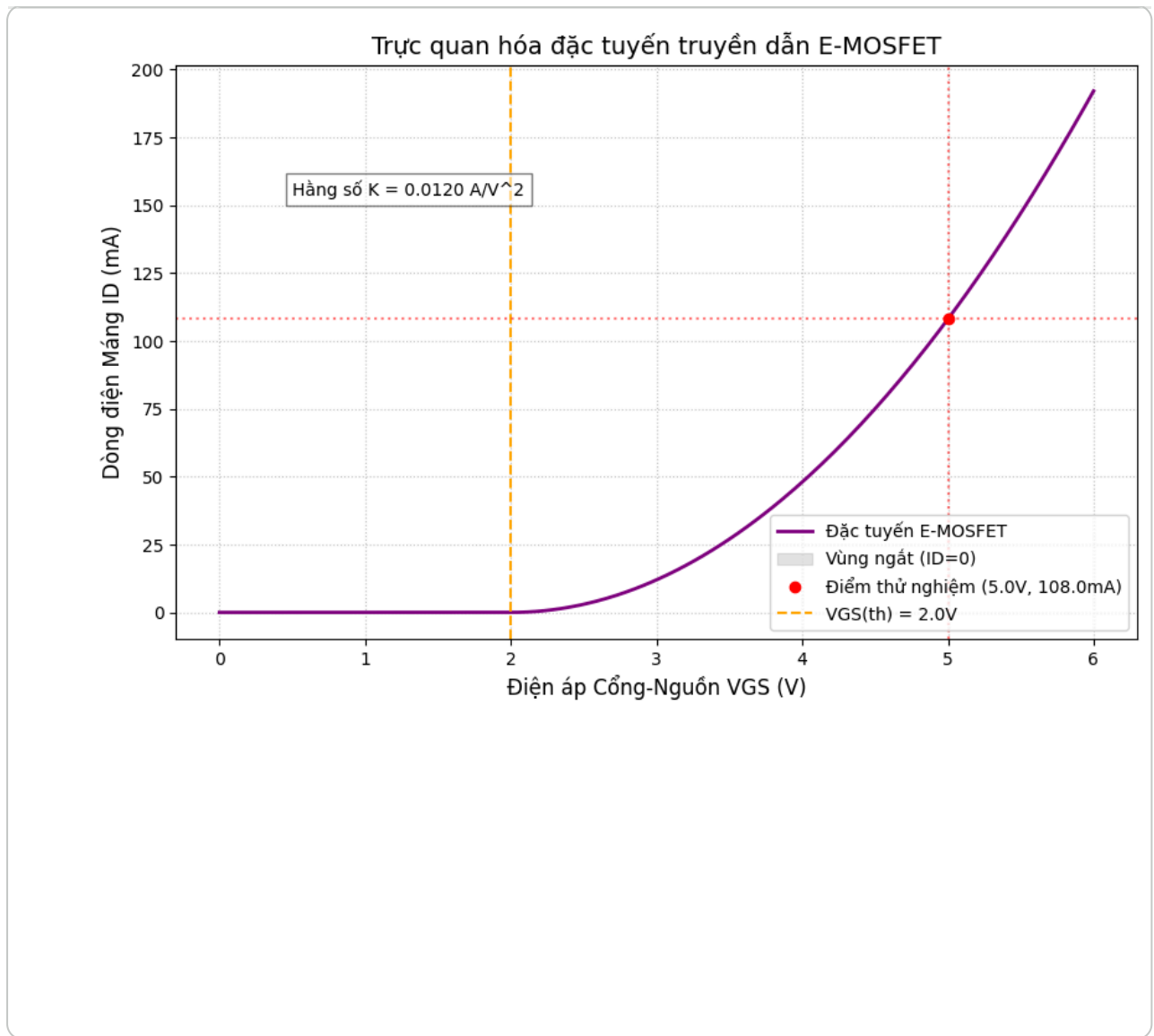
# Đánh dấu điểm ngưỡng VGS(th)
plt.axvline(x=vgs_threshold, color='orange', linestyle='--', label=f'VGS(th)

# Vẽ các đường giống
plt.axhline(y=id_test * 1000, color='red', linestyle=':', alpha=0.5)
plt.axvline(x=vgs_test, color='red', linestyle=':', alpha=0.5)

# Tùy chỉnh đồ thị
plt.title('Trực quan hóa đặc tuyến truyền dẫn E-MOSFET', fontsize=14)
plt.xlabel('Điện áp Cổng-Nguồn VGS (V)', fontsize=12)
plt.ylabel('Dòng điện Máng ID (mA)', fontsize=12)
plt.grid(True, linestyle=':', alpha=0.7)
plt.legend()

# Chú thích về hằng số K
plt.text(0.5, (max(id_range)*1000)*0.8, f'Hằng số K = {k_val:.4f} A/V^2',
        bbox=dict(facecolor='white', alpha=0.5))

plt.show()
```



Chương trình mẫu 4:

Phân tích vùng Ohm (Ohmic Region) Tính điện trở máng-nguồn (r_{ds}): Sử dụng JFET như một điện trở thay đổi được bằng điện áp, tính toán giá trị điện trở dựa trên V_{GS} .

- ✓ (Troubleshooting) Lập trình mô phỏng các lỗi mạch: Ví dụ như tính toán sự thay đổi của điện áp máng (V_D) khi một điện trở bị hở mạch hoặc ngắn mạch. Ví

dụ cụ thể từ tài liệu: Ví dụ 8-4: Xác định độ hỗ dẫn gm cho JFET 2N5457 tại $V_{GS} = -4V$.

Để hoàn tất bộ công cụ lập trình giải bài tập trong tài liệu của Floyd, dưới đây là mã Python tập trung vào vùng Ohmic, xử lý lỗi (Troubleshooting) và giải chi tiết Ví dụ 8-4.1. Phân tích vùng Ohmic và Điện trở Máng - Nguồn (r_{ds}) Trong vùng Ohmic ($V_{DS} < V_P$), JFET hoạt động như một điện trở tuyến tính được điều khiển bởi điện áp V_{GS} .

```
def calculate_rds(rds_on, vgs, vgs_off):
    """
    Tính điện trở máng-nguồn (rds) của JFET trong vùng Ohmic.
    Công thức: rds = rds(on) / (1 - VGS/VGS_off)
    rds(on) là điện trở khi VGS = 0.
    """
    if vgs == vgs_off:
        return float('inf') # Điện trở vô hạn khi kênh bị ngắt

    # Đảm bảo VGS không vượt quá điểm ngắt vật lý
    if abs(vgs) > abs(vgs_off):
        return float('inf')

    rds = rds_on / (1 - vgs / vgs_off)
    return rds

# Ví dụ áp dụng:
rds_0 = 100 # rds(on) = 100 Ohm khi VGS = 0
vgs_off_val = -7.0
vgs_target = -3.5

resistance = calculate_rds(rds_0, vgs_target, vgs_off_val)
print(f"Điện trở rds tại VGS = {vgs_target}V là: {resistance:.2f} Ohm")
```

Điện trở rds tại VGS = -3.5V là: 200.00 Ohm

✓ Biểu diễn trực quan:

Để trực quan hóa hàm tính điện trở máng-nguồn (r_{ds}) trong vùng Ohmic, chúng ta sẽ vẽ đồ thị biểu diễn sự thay đổi của điện trở khi điện áp V_{GS} tiến dần về giá trị ngắt ($V_{GS(off)}$). Trong vùng này, JFET hoạt động như một điện trở điều khiển bằng điện áp (voltage-controlled resistor). Khi V_{GS} càng âm (tiến về $V_{GS(off)}$), kênh dẫn càng hẹp lại và điện trở sẽ tăng vọt lên vô hạn. Dưới đây là mã Python sử dụng numpy và matplotlib:

```

import numpy as np
import matplotlib.pyplot as plt

def calculate_rds(rds_on, vgs, vgs_off):
    if abs(vgs) >= abs(vgs_off):
        return float('inf')
    return rds_on / (1 - vgs / vgs_off)

# --- Thông số ---
rds_0 = 100
vgs_off_val = -7.0
vgs_target = -3.5
res_target = calculate_rds(rds_0, vgs_target, vgs_off_val)

vgs_values = np.linspace(0, vgs_off_val + 0.5, 100)
rds_values = [calculate_rds(rds_0, v, vgs_off_val) for v in vgs_values]

plt.figure(figsize=(10, 6))
plt.plot(vgs_values, rds_values, label='Đặc tuyến  $r_{ds}$  vs  $V_{GS}$ ', col

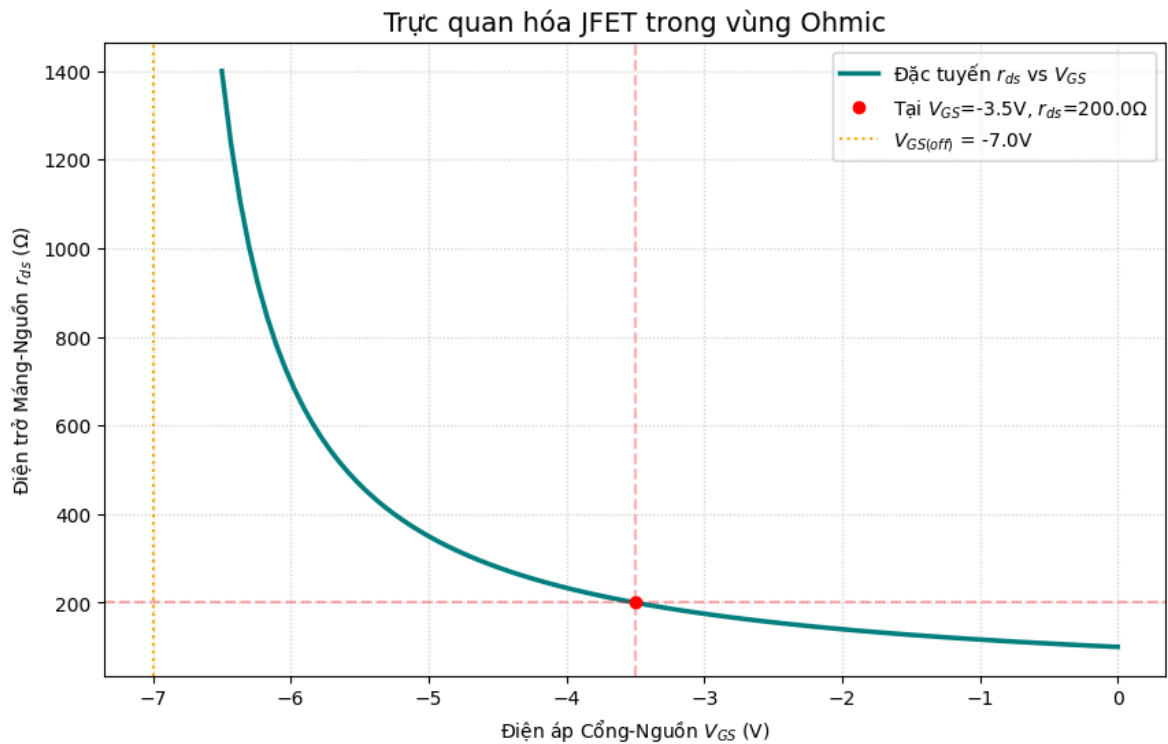
# --- SỬA LỖI TẠI ĐÂY ---
# Cách 1: Dùng {{ }} để thoát chuỗi f-string cho LaTeX
plt.plot(vgs_target, res_target, 'ro',
        label=f'Tại  $V_{GS}={vgs\_target}V$ ,  $r_{ds}={res\_target:.1f}\Omega$ '

# Hoặc Cách 2: Viết đơn giản không dùng dấu ngoặc nhọn trong LaTeX
# label=f'Tại  $V_{GS}={vgs\_target}V$ ,  $r_{ds}={res\_target:.1f}\Omega$ '

plt.axvline(x=vgs_target, color='red', linestyle='--', alpha=0.3)
plt.axhline(y=res_target, color='red', linestyle='--', alpha=0.3)
plt.axvline(x=vgs_off_val, color='orange', linestyle=':', label=f' $V_{GS}(of$ 

plt.title('Trực quan hóa JFET trong vùng Ohmic', fontsize=14)
plt.xlabel('Điện áp Cổng-Nguồn  $V_{GS}$  (V)')
plt.ylabel('Điện trở Máng-Nguồn  $r_{ds}$  ( $\Omega$ )')
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend()
plt.show()

```



2. Mô phỏng xử lý lỗi (Troubleshooting) Phần này giúp bạn tính toán sự thay đổi điện áp khi các linh kiện trong mạch bị hỏng (hở mạch hoặc ngắn mạch).

```
def troubleshoot_v_drain(vdd, rd, id_q, fault_type=None):
    """
    Mô phỏng điện áp máng VD dựa trên tình trạng linh kiện.
    Mạch giả định: VDD nối tiếp RD, JFET, và RS xuống đất.
    """
    if fault_type == "RD_OPEN":
        # Điện trở RD bị hở: Không có dòng điện, VD = 0V (tùy vị trí đo)
        return 0.0
    elif fault_type == "RD_SHORT":
        # Điện trở RD bị ngắn mạch: VD nối trực tiếp vào VDD
        return vdd
    elif fault_type == "JFET_OPEN":
        # JFET bị hỏng hở: Không có dòng qua RD, áp rơi trên RD = 0
        return vdd
    else:
        # Trạng thái bình thường
        return vdd - id_q * rd

# Ví dụ mô phỏng lỗi
```

```
VDD_val = 12.0
RD_val = 1.5e3 # 1.5k Ohm
ID_normal = 2e-3 # 2mA

print(f"VD Bình thường: {troubleshoot_v_drain(VDD_val, RD_val, ID_normal):.2}
print(f"VD khi RD bị hở: {troubleshoot_v_drain(VDD_val, RD_val, ID_normal, '
print(f"VD khi JFET bị hở: {troubleshoot_v_drain(VDD_val, RD_val, ID_normal,
```

```
VD Bình thường: 9.00 V
VD khi RD bị hở: 0.00 V
VD khi JFET bị hở: 12.00 V
```

✓ Biểu diễn trực quan:

Để biểu diễn trực quan các kịch bản xử lý lỗi (Troubleshooting) trong mạch JFET, cách tốt nhất là sử dụng biểu đồ cột (Bar chart). Điều này giúp bạn dễ dàng so sánh sự thay đổi của điện áp máng (V_D) giữa trạng thái hoạt động bình thường và các trạng thái lỗi phổ biến. Dưới đây là mã Python sử dụng matplotlib để trực quan hóa hàm `troubleshoot_v_drain`:

```
import matplotlib.pyplot as plt

def troubleshoot_v_drain(vdd, rd, id_q, fault_type=None):
    """Mô phỏng điện áp máng VD dựa trên tình trạng linh kiện."""
    if fault_type == "RD_OPEN":
        return 0.0
    elif fault_type == "RD_SHORT":
        return vdd
    elif fault_type == "JFET_OPEN":
        return vdd
    else:
        return vdd - id_q * rd

# --- Cấu hình thông số ---
VDD_val = 12.0
RD_val = 1.5e3 # 1.5k Ohm
ID_normal = 2e-3 # 2mA

# Danh sách các kịch bản lỗi
scenarios = {
    "Bình thường": None,
    "RD bị hở\n(RD_OPEN)": "RD_OPEN",
    "RD ngắn mạch\n(RD_SHORT)": "RD_SHORT",
    "JFET bị hở\n(JFET_OPEN)": "JFET_OPEN"
}

# Tính toán giá trị VD cho từng kịch bản
labels = list(scenarios.keys())
vd_values = [troubleshoot_v_drain(VDD_val, RD_val, ID_normal, scenarios[s])

# --- VẼ ĐỒ THỊ ---
```

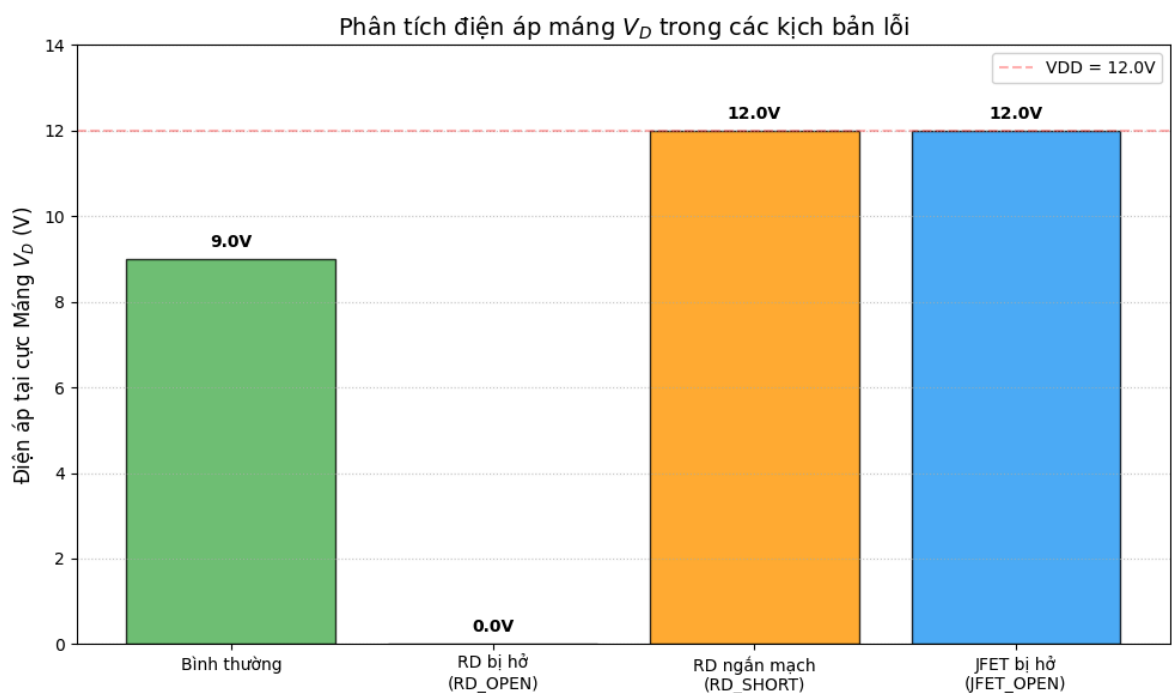
```
plt.figure(figsize=(10, 6))
colors = ['#4CAF50', '#F44336', '#FF9800', '#2196F3'] # Màu sắc khác nhau ch

bars = plt.bar(labels, vd_values, color=colors, edgecolor='black', alpha=0.8)

# Thêm giá trị cụ thể lên đầu mỗi cột
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.2, f'{yval:.1f}V',
             ha='center', va='bottom', fontweight='bold')

# Tùy chỉnh đồ thị
plt.axhline(y=VDD_val, color='red', linestyle='--', alpha=0.3, label=f'VDD =
plt.title('Phân tích điện áp máng $V_D$ trong các kịch bản lỗi', fontsize=14)
plt.ylabel('Điện áp tại cực Máng $V_D$ (V)', fontsize=12)
plt.ylim(0, VDD_val + 2) # Giới hạn trục Y để dễ nhìn
plt.grid(axis='y', linestyle=':', alpha=0.7)
plt.legend()

plt.tight_layout()
plt.show()
```



3. Giải Ví dụ 8-4 (Trang 411 trong tài liệu) **Đề bài:** Xác định độ dẫn g_m cho JFET 2N5457 tại $V_{GS} = -4V$. Biết từ bảng dữ liệu $I_{DSS} = 3.0\text{ mA}$ và $V_{GS(off)} = -6V$.

```
def solve_example_8_4():
    # Thông số đề bài
    idss = 3.0e-3      # 3.0 mA
    vgs_off = -6.0     # -6 V
    vgs_point = -4.0   # Điểm cần tính

    # 1. Tính gm0 (độ dẫn cực đại tại VGS = 0)
    # Công thức: gm0 = 2 * IDSS / |VGS(off)|
    gm0 = (2 * idss) / abs(vgs_off)

    # 2. Tính gm tại VGS = -4V
    # Công thức: gm = gm0 * (1 - VGS / VGS(off))
    gm = gm0 * (1 - vgs_point / vgs_off)

    return gm0, gm

# Chạy giải ví dụ
gm0, gm_at_4v = solve_example_8_4()
print(f"--- GIẢI VÍ DỤ 8-4 ---")
print(f"Độ dẫn cực đại gm0: {gm0*1e6:.0f} uS")
print(f"Độ dẫn tại VGS = -4V: {gm_at_4v*1e6:.0f} uS")

--- GIẢI VÍ DỤ 8-4 ---
Độ dẫn cực đại gm0: 1000 uS
Độ dẫn tại VGS = -4V: 333 uS
```

✓ Biểu diễn trực quan: